PHASE 5 – PROJECT DEVELOPMENT

AI FAQ ASSISTANT API

AI-Powered FAQ and Customer Support Automation

---

1. INTRODUCTION

Project Development is the implementation phase of the **AI FAQ Assistant API**.

Based on the requirements and design completed in the previous phases, the backend application was developed using **Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, and Google Gemini AI**.

The project follows a modular MVC-based structure. The implementation separates routes, controllers, middleware, models, services, configuration, and utility functions.

The completed source code contains the backend application, database models, authentication functionality, FAQ management, FAQ search, Google Gemini integration, API testing script, and Postman collection.

---

2. DEVELOPMENT ENVIRONMENT

The project was developed using the following environment:

Component	**Technology**
Programming Language	JavaScript
Runtime	Node.js
Backend Framework	Express.js
Database	MongoDB
ODM	Mongoose
Authentication	JWT
Password Hashing	bcrypt
AI Service	Google Gemini
AI SDK	@google/genai

Component	Technology
API Testing	Postman
Code Editor	Visual Studio Code
Package Manager	npm

The project requires Node.js 18+ or 20+, MongoDB or MongoDB Atlas, and a Google Gemini API key.

---

3. PROJECT STRUCTURE

The backend was organized into separate folders according to their responsibilities.

Folder/File	Purpose
config/	Database configuration
controllers/	Application logic
middleware/	Authentication, validation and error handling
models/	MongoDB schemas
routes/	API route definitions
services/	Google Gemini integration
utils/	Helper functions
app.js	Express application configuration
server.js	Server startup
test_api.js	Automated API testing
package.json	Dependencies and scripts
AI_FAQ_Assistant_API.postman_collection.json	Postman API collection

Suggested Screenshot

Figure 5.1 – Project Folder Structure

---

4. BACKEND SETUP

The Node.js project was initialized and the required dependencies were installed using npm.

The major dependencies include:

- Express
- Mongoose
- JSON Web Token
- bcrypt
- dotenv
- CORS
- Google GenAI SDK

The Express application was configured to:

- Accept JSON requests.
- Enable CORS.
- Register API routes.
- Apply middleware.
- Handle errors.

The server is configured to run on port **5000** by default.

Development Command:

npm run dev

Production Command:

npm start

---

5. DATABASE IMPLEMENTATION

MongoDB was implemented as the primary database.

Mongoose was used to create and manage the application data models.

The current implementation contains two main models:

- User
- FAQ

5.1 User Model

The User model stores account information.

Field	Purpose
name	Stores user name
email	Stores unique email
password	Stores hashed password
createdAt	Account creation time
updatedAt	Account update time

Passwords are hashed using bcrypt before being stored.

5.2 FAQ Model

The FAQ model stores FAQ information.

Field	Purpose
question	FAQ question
answer	FAQ answer
category	FAQ category
createdBy	User who created the FAQ
createdAt	FAQ creation time
updatedAt	FAQ update time

The available FAQ categories are:

- Technology
- Education
- Health
- Banking

- General

---

6. USER AUTHENTICATION IMPLEMENTATION

The authentication module was developed to provide secure user access.

The module contains:

Registration

The registration process:

1. Receives user information.
2. Validates the request.
3. Checks for an existing email.
4. Hashes the password using bcrypt.
5. Creates the user.
6. Returns the registration response.

Login

The login process:

1. Receives email and password.
2. Finds the user.
3. Compares the password using bcrypt.
4. Generates a JWT token.
5. Returns the user information and token.

Profile

The profile endpoint is protected using JWT authentication and returns information about the currently authenticated user.

Authentication Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user
GET	/api/auth/profile	Get authenticated profile

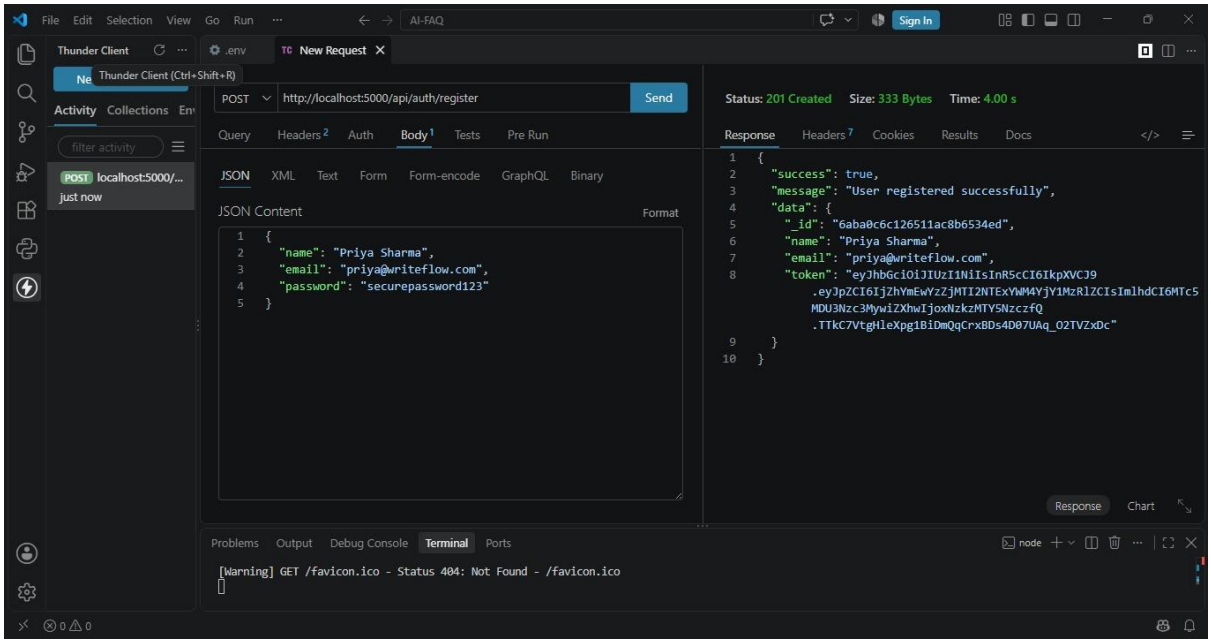


Figure – User Registration API Implementation

---

7. FAQ MANAGEMENT IMPLEMENTATION

The FAQ module provides complete CRUD functionality.

Create FAQ

Authenticated users can create a FAQ by providing:

- Question
- Answer
- Category

The user ID is automatically stored as the creator.

Read FAQ

Users can:

- View all FAQs.
- View an individual FAQ.

Update FAQ

The creator of an FAQ can update its information.

The system checks the authenticated user's ID against the createdBy field before allowing modification.

Delete FAQ

The creator can delete the FAQ after ownership verification.

FAQ Endpoints

Method	Endpoint	Purpose
POST	/api/faqs	Create FAQ
GET	/api/faqs	Get all FAQs
GET	/api/faqs/:id	Get specific FAQ
PUT	/api/faqs/:id	Update FAQ
DELETE	/api/faqs/:id	Delete FAQ

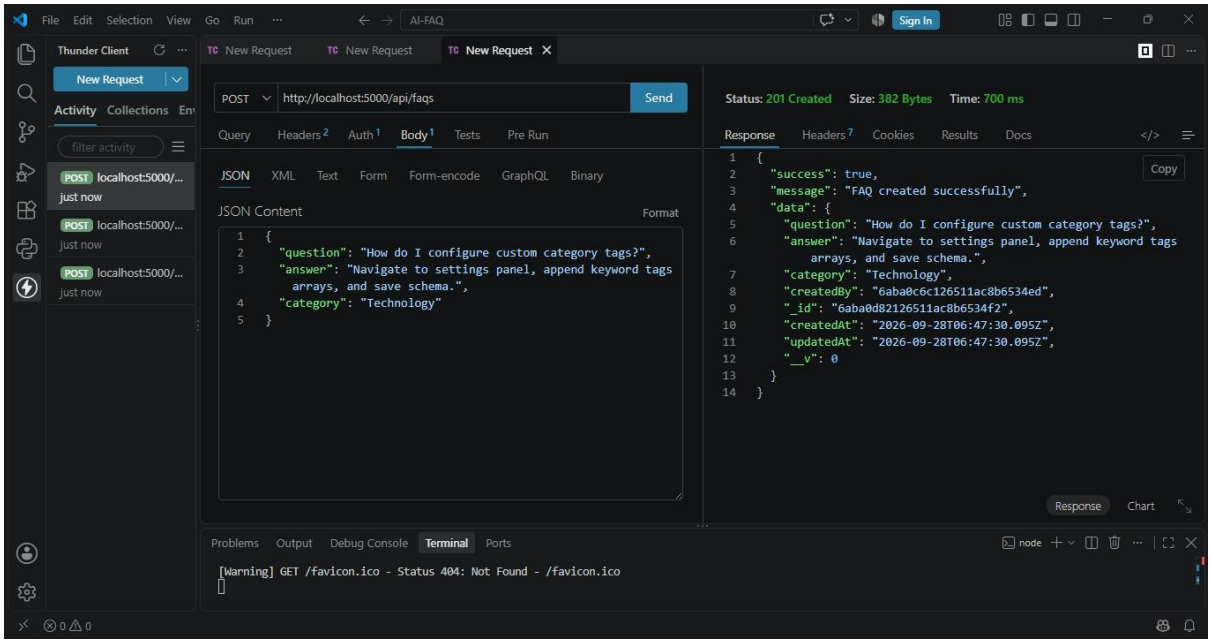


Figure – FAQ Creation API Implementation

---

8. FAQ SEARCH IMPLEMENTATION

A search facility was implemented for retrieving relevant FAQ records.

The search endpoint accepts a query parameter:

GET /api/faqs/search?q=query_string

The implementation searches across:

- FAQ question
- FAQ answer
- FAQ category

The current implementation uses MongoDB regular-expression matching for keyword-based searching.

This provides a simple method for finding FAQ records without requiring the user to browse the entire collection.

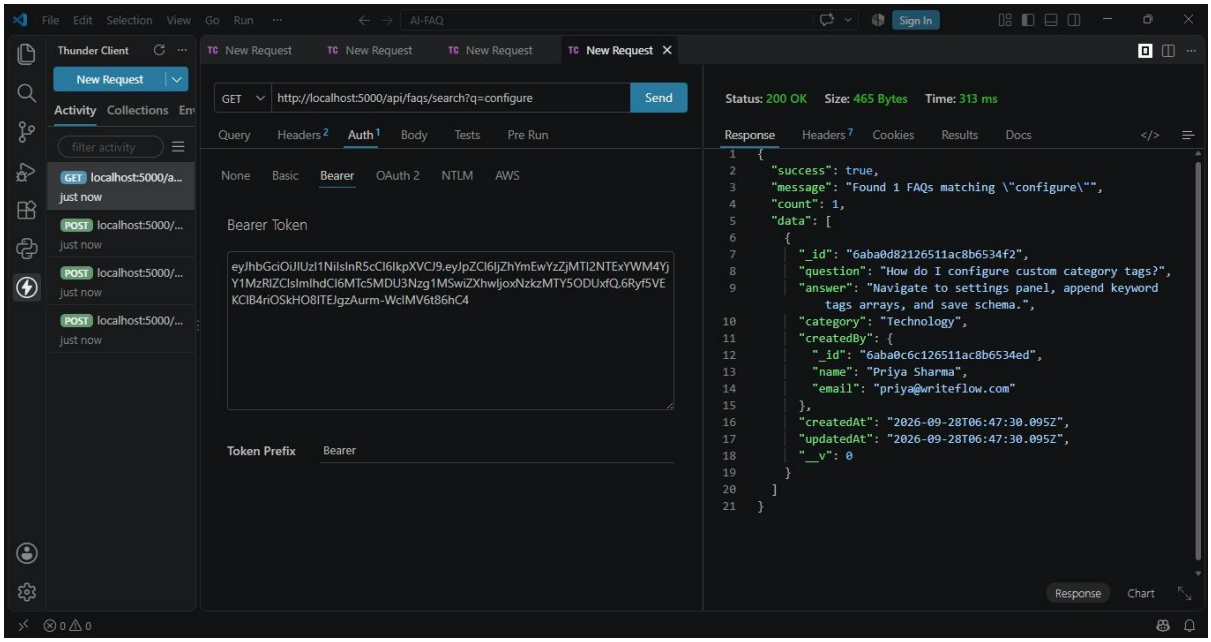


Figure – FAQ Search API Implementation

---

9. GOOGLE GEMINI AI IMPLEMENTATION

Google Gemini was integrated into the backend using the official **Google GenAI SDK**.

The AI functionality is implemented through a separate Gemini service.

Two major AI features were developed.

9.1 AI Answer Generator

The AI Answer endpoint accepts a question and sends it to Google Gemini.

Endpoint

POST /api/ai/answer

Input

{

"question": "What is Artificial Intelligence?"

}

The generated answer is returned through the API.

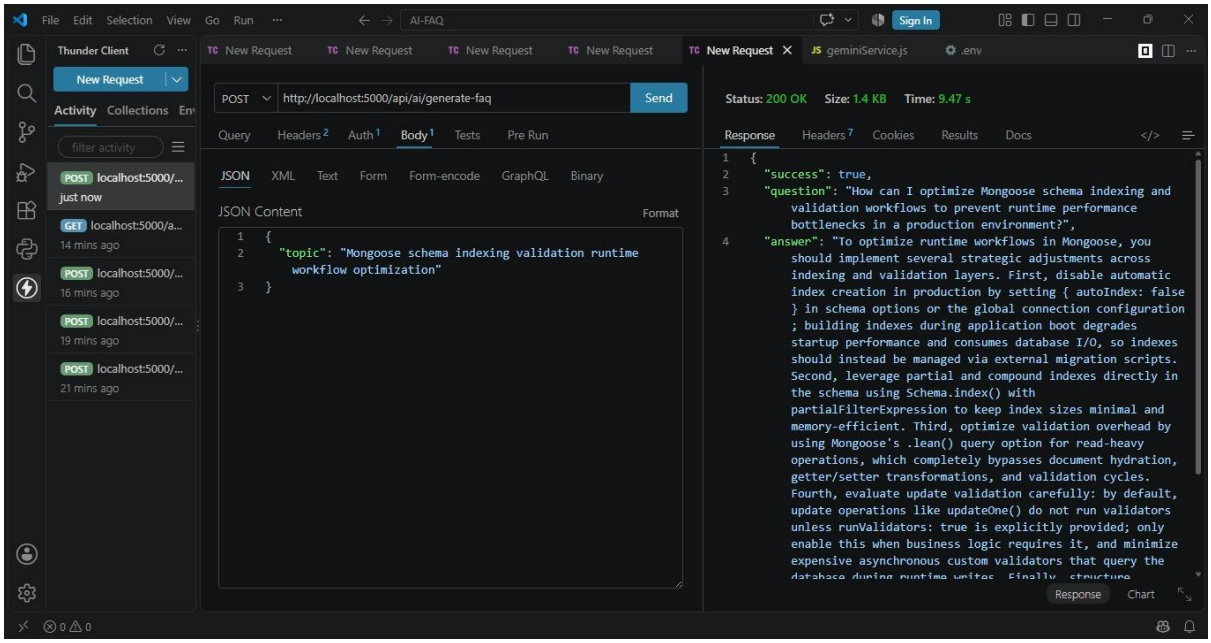


Figure – AI FAQ Generation API Implementation

9.2 AI FAQ Generator

The AI FAQ Generator accepts a topic and generates a structured FAQ pair.

Endpoint

POST /api/ai/generate-faq

Input

{

"topic": "MongoDB"

}

Output

The response contains:

- Generated question
- Generated answer

The implementation uses structured JSON output so that the generated FAQ can be returned in a predictable format.

The generated question-answer pair can then be submitted to the FAQ creation endpoint for storage.

---

10. SECURITY AND ERROR HANDLING IMPLEMENTATION

Security-related functionality was implemented using middleware and utility components.

JWT Authentication

Protected API requests require a valid JWT token.

Password Protection

User passwords are hashed using bcrypt before database storage.

Input Validation

Request data is validated before being processed by the controllers.

Ownership Verification

FAQ update and delete operations verify that the authenticated user is the creator of the FAQ.

Error Handling

A centralized error middleware handles different types of application errors, including:

- Validation errors
- Invalid MongoDB IDs
- Duplicate database values
- Authentication errors
- General server errors

This provides consistent error responses to API clients.

---

11. API IMPLEMENTATION SUMMARY

The completed API contains the following major endpoints:

Module	Method	Endpoint
Authentication	POST	/api/auth/register
Authentication	POST	/api/auth/login
Authentication	GET	/api/auth/profile
FAQ	POST	/api/faqs
FAQ	GET	/api/faqs
FAQ	GET	/api/faqs/:id
FAQ	PUT	/api/faqs/:id
FAQ	DELETE	/api/faqs/:id
Search	GET	/api/faqs/search?q=
AI	POST	/api/ai/answer
AI	POST	/api/ai/generate-faq

These endpoints form the main functional interface of the AI FAQ Assistant API.

---

12. POSTMAN INTEGRATION

A Postman collection was included with the project for API testing.

The collection contains the major authentication, FAQ, search, and AI endpoints.

The collection uses:

http://localhost:5000

as the default base URL.

A token variable is used for protected API requests. The login request can automatically save the JWT token so that subsequent protected requests can use it.

---

13. AUTOMATED API TEST IMPLEMENTATION

An automated API test file named:

test_api.js

is included in the project.

The test implementation covers major operations including:

- API health check
- User registration
- Duplicate registration
- Login
- Profile access
- FAQ creation
- FAQ retrieval
- FAQ search
- FAQ update
- AI answer
- AI FAQ generation
- FAQ deletion
- Invalid input handling

This test file provides an additional method of verifying the functionality of the backend.

Detailed testing results will be documented separately in **Phase 6 – Project Testing**.

14. DEVELOPMENT OUTPUT

The development phase produced a functional backend containing:

Component	**Development Status**
Express Server	Completed
MongoDB Connection	Completed
User Model	Completed
FAQ Model	Completed
Registration	Completed

Component	Development Status
Login	Completed
JWT Authentication	Completed
Profile API	Completed
FAQ CRUD	Completed
FAQ Search	Completed
Gemini AI Answer	Completed
Gemini FAQ Generation	Completed
Validation Middleware	Completed
Error Middleware	Completed
Postman Collection	Completed
API Test Script	Completed

15. DEVELOPMENT CONCLUSION

The **AI FAQ Assistant API** was implemented as a modular Node.js and Express.js backend.

The development phase successfully combined MongoDB-based FAQ management with secure authentication and Google Gemini AI functionality.

The implemented system supports user registration and login, protected API access, FAQ CRUD operations, keyword-based FAQ search, AI-generated answers, and AI-generated FAQ content.

The project source code is organized into separate modules for routes, controllers, middleware, models, services, configuration, and utilities. This provides a structured foundation for testing and future development.

The next phase, **Phase 6 – Project Testing**, will verify the implemented functionality using Postman and the automated API test script, with screenshots of successful and failed test cases.